🏛️ COMPREHENSIVE TECHNICAL & ARCHITECTURAL BLUEPRINT

Djerba Delivery — Everything Explained From Scratch

A complete, foundational, and code-level walkthrough explaining languages, runtimes, frameworks, APIs, databases, environments, and every single tool in the platform.

`Dart & Flutter 3` `JS vs TypeScript` `Node.js Runtime` `Next.js 16 + Hono 4` `Drizzle ORM Translator` `SQLite Permanent Vault` `Redis Mutex Lock` `Better-Auth & RBAC` `OpenStreetMap GPS` `Firebase FCM Pager` `Pollinations Generative AI`

4 Apps
CLIENT · DRIVER · RESTO · ADMIN

1 Engine
NEXT.JS + HONO BACKEND

< 1 ms
IN-MEMORY ROUTE DISPATCH

100% GPS
ZERO POSTAL ADDRESS NEEDED

DOCUMENT TARGET
Executives, Students & Non-Developers

PROJECT TERRITORY
Djerba Island (514 km² Mediterranean Territory)

EDITION
Comprehensive Master Guide 2026

01. Core Computing Fundamentals (From Scratch)

1. What is a Programming Language?

A programming language is a structured vocabulary and grammar that humans use to write instructions for computers. In our project, we use **3 distinct languages**:

- **Dart:** Google's language used to paint visual buttons, maps, and screens on phones (Flutter).
- **TypeScript / JavaScript:** The mathematical calculation language used on our backend server.
- **SQL (Structured Query Language):** The universal database language used to fetch and store records.

2. JavaScript (JS) vs. TypeScript (TS) — Why Do Both Exist?

📜 **JavaScript (JS): The Flexible Pioneer**

Created in 1995, JS has no strict data types. You can put numbers into a text variable without warning. While fast to write, silly typos and type errors crash the application in front of real users.

🛡️ **TypeScript (TS): JavaScript with Armor**

Created by Microsoft, TypeScript adds strict **Types** (e.g. `price: number`). If you attempt to assign text to a price, **the compiler highlights it in red and refuses to build**, catching bugs before release!

3. What is Node.js and What is a "Runtime / Environment"?

A language is just text. To execute it, you need a **Runtime Environment**—a software engine that reads lines of code and translates them into CPU machine instructions (1s and 0s).

- **The Historical Constraint (1995–2009):** JavaScript could ONLY run inside web browsers (Chrome, Firefox). It could only animate HTML buttons. It was completely forbidden from touching files or databases.
- **The Node.js Breakthrough (2009):** Ryan Dahl extracted Google Chrome's V8 engine into a standalone computer program called **Node.js**.
- **The Result:** JavaScript gained the ability to run on Windows/Linux servers, query databases, read disk files, and listen on network ports. Today, TypeScript/Node.js powers server infrastructure at **Netflix, Uber, PayPal, and Airbnb**!

4. What are Environment Variables (`.env`)?

Your server needs secret credentials (database passwords, SMS API keys, session tokens). **You must never write passwords directly in code files** because publishing code to GitHub would expose them to hackers. Instead, your server reads a hidden file named `.env` (short for *Environment*) on startup. The values stay safely in computer memory.

5. Framework vs. Library: What's the Difference?

> 💡 **THE TOOLBOX VS. THE HOUSE**
>
> - **A Library (Toolbox):** A specific tool you call when you need it (e.g., `Dio` for HTTP requests, `Zod` for validation). You remain in full control of your application flow.
> - **A Framework (House):** A pre-built structure with strict architectural rules (e.g., **Flutter** for mobile, **Next.js** for servers). The framework controls the execution flow; you simply write your code in designated folders.

6. What is an API (Application Programming Interface)?

An API is a **digital waiter** connecting separate software applications.

When you sit at a restaurant (the **Client Mobile App**), you cannot walk into the kitchen yourself (security reasons!). Instead, you give your order to the **Waiter (the API)**. The waiter carries the order to the chef (the **Backend Server & Database**) and brings your food back.

7. The Universal Language Rule: Why Dart + TypeScript?

The mobile app (written in **Dart**) and the backend (written in **TypeScript**) do NOT need the same language! They communicate across the internet using **JSON (JavaScript Object Notation)**—plain text formatted like: `{"meal": "Couscous", "price": 18.500, "lat": 33.8869}`. Both Dart and TypeScript read and write JSON natively!

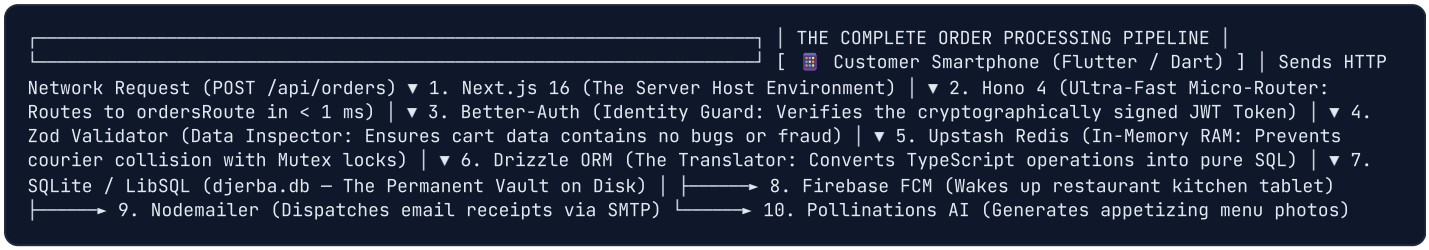
02. The Frontend Stack (Flutter & Dart Ecosystem)

The frontend is everything the user sees, touches, and interacts with on screen. In our workspace, there are 4 dedicated frontends: **djerba_client_app**, **djerba_driver_app**, **djerba_restaurant_dashboard**, and **djerba_admin_dashboard**.

Frontend Tool	Exact Responsibility	Why It Was Chosen	Real-World Analogy
Dart	Client-side programming language.	Compiles directly to native ARM machine code on smartphones, delivering butter-smooth **60–120 FPS** screen animations.	The master painter's specialized brush.
Flutter 3	Google's cross-platform UI framework.	**Write Once, Run Everywhere:** Compiles the exact same codebase to iOS, Android, and Web simultaneously, avoiding separate development teams.	An architectural blueprint that builds in brick, wood, or steel automatically.
Riverpod	Reactive state management engine.	Keeps cart items, login status, and active deliveries synchronized across all screens without memory leaks or page reloads.	A central stock ticker updating all wall displays simultaneously.
GoRouter	Declarative URL navigation router.	Controls smooth transitions between screens (e.g., `/login` ➔ `/home` ➔ `/tracking`) and manages browser deep links.	The directional signs guiding travelers through an airport.
Dio	Advanced HTTP networking client.	Packs requests, attaches `Authorization: Bearer <token>` headers automatically, and handles network retries if 4G drops.	The armored courier van carrying encrypted documents to the server.
FlutterMap	Open-source map tile renderer.	**Zero Google Maps fees:** Google charges up to $7 per 1,000 map loads. FlutterMap uses OpenStreetMap at zero cost with high GPS accuracy.	A high-resolution laminated map book that never charges subscription fees.
Geolocator	Hardware device GPS bridge.	Directly reads real-time latitude, longitude, heading, and speed from the delivery courier's smartphone GPS chip.	The digital satellite speedometer mounted on the courier's motorcycle.
FL_Chart	Hardware-accelerated charting library.	Renders interactive financial line and bar charts inside the Admin Dashboard for revenue, volume, and courier rankings.	A financial analyst's live Wall Street trading terminal.

03. The Backend Stack (The 10 Tools in djerba_backend)

The backend is the unseen central engine running 24/7 on a server. It has no user interface; it processes calculations, coordinates actors, and manages the database.



Backend Tool	Exact Responsibility	Why It Is Essential	Real-World Analogy
Next.js 16	Server runtime and compiler.	Provides unified compilation, environment management, and production cloud packaging.	The commercial delivery truck chassis.
Hono 4	High-speed API micro-router.	Routes HTTP network requests in **< 1 ms** instead of 15 ms with legacy frameworks.	The high-speed automated mail sorting conveyor.
Better-Auth	Authentication and session suite.	Hashes passwords (bcrypt) and generates tamper-proof JWT Bearer session tokens.	The passport office stamping verified visas.
Zod	Strict data schema validator.	Rejects corrupted or malicious data (e.g. negative prices, missing coordinates) before execution.	The security guard checking tickets and inspecting bags.
Upstash Redis	In-memory RAM data store.	Provides sub-millisecond **Mutex Locks** to prevent couriers from double-claiming runs.	The magnetic board where active courier badges are moved live.
Drizzle ORM	TypeScript-to-SQL translator.	Eliminates typo bugs in database columns and prevents SQL injection vulnerabilities.	The certified translator turning contracts into legal Latin.
SQLite (djerba.db)	Relational database file on disk.	Permanently stores users, restaurants, orders, and financial balances with bank-level ACID reliability.	The heavy fireproof safe inside the headquarters.
Firebase Admin	Mobile push notifications (FCM).	Rings courier and kitchen phones even when their apps are closed in the background.	An emergency pocket alert pager.
Nodemailer	SMTP email dispatcher.	Sends password reset OTP codes and receipts through Gmail SMTP.	The postal courier delivering certified letters.
Pollinations AI	Generative food photography proxy.	Automatically creates appetizing, photorealistic dish pictures for restaurant menus on demand.	An on-call food photographer generating catalog art in 3 seconds.

04. Annotated Code Walkthroughs (Real Production Code)

1. Database Schema & Price Snapshotting (schema.ts)

djerba_backend/src/db/schema.ts — Order & Items Table — Drizzle ORM & TypeScript

```
export const orders = sqliteTable('orders', { id: text('id').primaryKey(), // Globally unique random fingerprint (UUID) orderNumber: text('order_number').unique().notNull(), // CMD-839201 clientId: text('client_id').references(() => user.id).notNull(), restaurantId: text('restaurant_id').references(() => restaurants.id).notNull(), driverId: text('driver_id').references(() => user.id), // Nullable until a courier claims it! status: text('status', { enum: ['pending', 'accepted', 'preparing', 'ready_for_pickup', 'picked_up', 'arrived_at_place', 'delivered', 'cancelled'] }).notNull().default('pending'), subtotal: real('subtotal').notNull(), deliveryFee: real('delivery_fee').notNull(), // Flat 4.500 TND totalAmount: real('total_amount').notNull(), deliveryAddress: text('delivery_address').notNull(), deliveryLat: real('delivery_lat').notNull(), // 33.8869 deliveryLng: real('delivery_lng').notNull(), // 10.8560 }); export const orderItems = sqliteTable('order_items', { id: text('id').primaryKey(), orderId: text('order_id').references(() => orders.id, { onDelete: 'cascade' }).notNull(), productId: text('product_id').references(() => products.id).notNull(), // PRICE SNAPSHOTTING: Frozen forever at checkout to survive future menu price hikes productNameSnapshot: text('product_name_snapshot').notNull(), unitPriceSnapshot: real('unit_price_snapshot').notNull(), quantity: integer('quantity').notNull(), });
```

2. Pricing Calculation & Atomic Transaction (orders.ts)

djerba_backend/src/server/routes/orders.ts — Server-Side Pricing — Anti-Tampering & Atomicity

```
// 1. DISCARD client prices! Query verified prices from SQLite database: const productIds = body.items.map(i => i.productId); const dbProducts = await db.select().from(products).where(inArray(products.id, productIds)); let subtotal = 0; for (const item of body.items) { const dbProduct = dbProducts.find(p => p.id === item.productId); subtotal += dbProduct.price * item.quantity; } const deliveryFee = 4.500; // Flat Djerba fee const totalAmount = subtotal + deliveryFee; // 2. ATOMIC TRANSACTION: Both order and items are saved together, or nothing is await db.transaction(async (tx) => { await tx.insert(orders).values({ ... }); await tx.insert(orderItems).values(itemsWithOrderId); });
```

3. The Redis Mutex Collision Guard (dispatch.ts)

djerba_backend/src/server/services/dispatch.ts — Atomic Lock — Concurrency Control

```
export class DispatchService { static async lockOrder(orderId: string, driverId: string): Promise<boolean> { // SET NX: "Set if Not eXists" (Succeeds only if key is currently free) // EX 10: "Expire in 10 seconds" (Auto-clears lock if driver phone disconnects) const acquired = await redis.set( `lock:order:${orderId}`, driverId, { nx: true, ex: 10 } ); return acquired !== null; } }
```

05. Financial Model & Multi-App Testing (Quad View)

The Financial Split (15% Platform Commission)

1. Meal Items (30.000 TND)
The restaurant receives **25.500 TND** (85%). The platform retains **4.500 TND** (15% commission fee).

2. Delivery Fee (4.500 TND)
A flat delivery fare paid by the customer for island transport, credited directly to the driver's earnings.

3. Cash on Delivery (COD)
Customer hands **34.500 TND** in cash to the courier. The courier's digital pouch records this cash balance for weekly reconciliation.

How to Test All 4 Apps Simultaneously (Quad View)

The included `quad_view.html` file allows you to test the entire platform lifecycle live on a single computer screen:
Step 1: In `djerba_backend`, launch the server: `npm run dev`
Step 2: Double-click `quad_view.html` to open it in Google Chrome or any browser.

Viewport Quadrant	Application	Local Port	What You Test Live
Top-Left	Customer App	`http://localhost:8080`	Browse food items, drop GPS pin on map, submit live order.
Top-Right	Restaurant Kitchen	`http://localhost:8081`	Listen to incoming chime, accept order, toggle to cooking.
Bottom-Left	Courier Driver	`http://localhost:8082`	Claim delivery run, view route navigation, tap "Delivered".
Bottom-Right	Super Admin	`http://localhost:8083`	Watch revenues, 15% commissions, and courier locations update in real time.

Master Vocabulary Cheat-Sheet

Term	What It Really Means
Dart	The compiled language used to write the mobile visual screens.
Flutter 3	The Google engine that paints buttons, animations, and maps on iOS, Android, and Web.
TypeScript (TS)	JavaScript with compile-time type-safety that prevents 95% of software bugs.
Node.js	The runtime engine that allows JavaScript/TypeScript to run outside browsers as a server.
Next.js 16	The full-stack framework hosting our server process and environment keys.
Hono 4	The ultra-fast micro-router sorting HTTP requests in under 1 millisecond.
API	The digital messenger carrying data between phones and the central server.
JSON	The universal plain text language used between Dart and TypeScript.
Drizzle ORM	The translator turning safe TypeScript operations into optimized SQL database commands.
SQLite (`djerba.db`)	The permanent relational filing cabinet on disk storing all users, menus, and orders.
Upstash Redis	In-memory RAM store used for real-time courier GPS tracking and collision-prevention locks.
Mutex Lock	The atomic rule guaranteeing that when two drivers tap "Accept", only one gets the delivery.